

PHP Deserialization





1. Giới thiệu về PHP Object Injection

- Bản chất của lỗi Injection là khiến cho hệ thống hiểu lầm user input là instruction.
- Lỗi này xảy ra khi kẻ xấu truyền vào một chuỗi đã được serialize vào hàm unserialize để gọi những hàm sink nguy hiểm (`system()`, `exec()`,...) trong chương trình.
- Điều đặc biệt của Object Injection là hậu quả của loại lỗi này rất đa dạng (vì nó phụ thuộc vào mẫu code có sẵn của chương trình).

1.1. Serialize

? Vậy serialize là gì?

- PHP (và rất nhiều ngôn ngữ khác) cung cấp cho người dùng một hàm gọi là `serialize()`
- Theo trang [php.net](https://www.php.net):

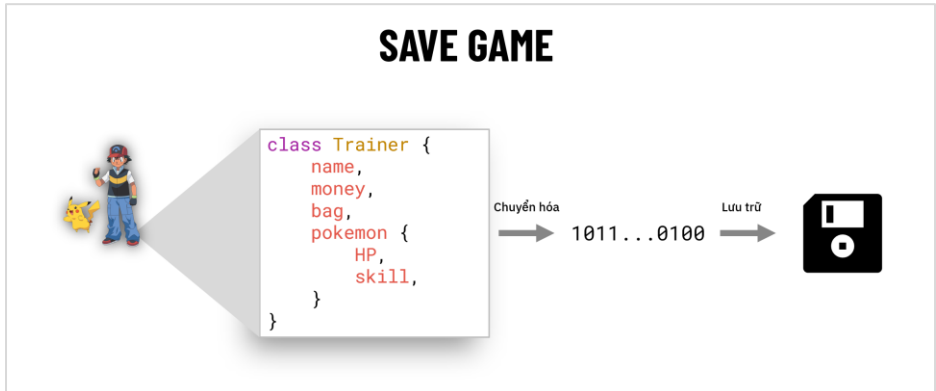
```
serialize(mixed $value): string
```

Generates a storable representation of a value.

This is useful for storing or passing PHP values around without losing their type and structure.

→ Hàm `serialize()` này sẽ chuyển hóa trạng thái của một đối tượng thành một chuỗi dữ liệu có thể lưu trữ hoặc truyền đi.

- Ví dụ quá trình Serialize được ứng dụng trong tính năng Save Game:



1.2. Unserialize

- Sau khi chuỗi dữ liệu đã được Serialize, phải có một cách nào đó để chuyển hóa chuỗi dữ liệu này về thành một trạng thái mà hệ thống có thể hiểu được.

→ Đây là nhiệm vụ của `unserialize()`.

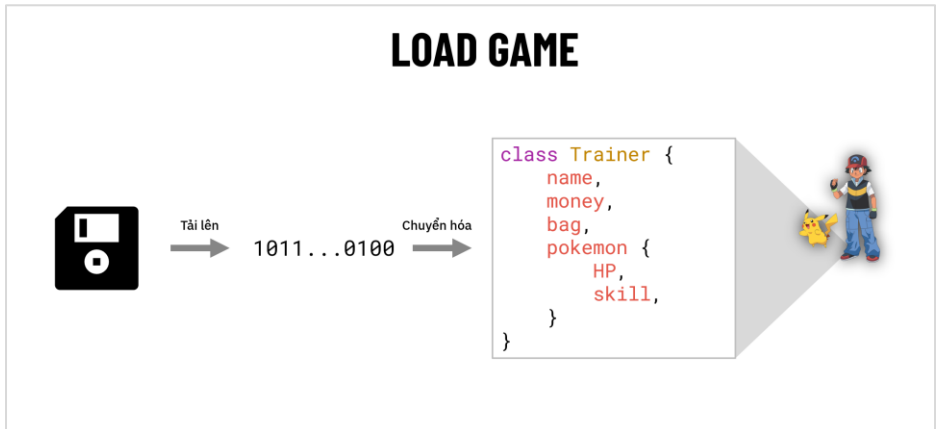
- Theo [php.net](https://www.php.net):

```
unserialize(string $data, array $options = []): mixed
```

`unserialize()` takes a single serialized variable and converts it back into a PHP value.

→ Hàm `unserialize()` sẽ chuyển hóa một chuỗi dữ liệu đã được serialize về value của PHP.

- Ví dụ quá trình unserialize được ứng dụng trong tính năng Load Game:



2. Khai thác PHP Object Injection

2.1. Magic Methods trong Object – Oriented Programming (OOP):

__construct()

- Hàm `__construct()` được sử dụng rất thường xuyên trong lập trình OOP.
- Hàm này giúp chúng ta khởi tạo thuộc tính của một đối tượng.
- PHP sẽ tự động gọi hàm này khi tạo đối tượng từ một lớp.
- Ví dụ: "Ash" được set vào thuộc tính name của lớp Trainer

```
class Trainer {  
    private $name;  
  
    public function __construct($name) {  
        $this->name = $name;  
    }  
}  
  
$a = new Trainer("Ash");
```

__destruct()

- Theo [php.net](https://www.php.net):

Destructor

```
__destruct(): void
```

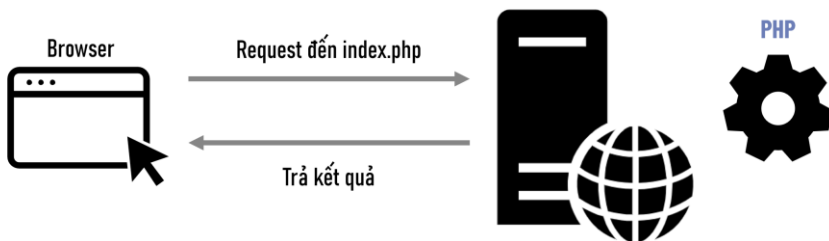
PHP possesses a destructor concept similar to that of other object-oriented languages, such as C++. The destructor method will be called as soon as there are no other references to a particular object, or in any order during the shutdown sequence.

- Hàm `__destruct()` sẽ được tự động được gọi khi đối tượng trong chương trình không còn được tham chiếu đến nữa.
- PHP sẽ tự động gọi mọi hàm `__destruct()` để hủy tất cả các class còn tồn tại sau khi hoàn tất chương trình.

→ Quá trình này gọi là quá trình Cleanup.

- PHP tiến vào quá trình Cleanup sau khi thực thi xong

PHP “TỰ HỦY” NHƯ THẾ NÀO?



- Tất cả các ngôn ngữ lập trình OOP đều có quá trình Cleanup.



__toString()

- `__toString()` sẽ được tự động gọi (với điều kiện phải định nghĩa hàm này trong class) khi muốn in một Object ra màn hình dưới dạng chuỗi.
- Ví dụ đoạn code định nghĩa `toString()` trong lớp Trainer:

```
src > 🐘 Trainer.php
1  <?php
2  class Trainer{
3      public $name;
4      public function __construct($name){
5          $this->name = $name;
6      }
7
8      public function __toString(){
9          return "Trainer: ".$this->name;
10     }
11 }
12
13 $trainer = new Trainer("Ash");
14 echo "#####\n";
15 echo "###.          .###\n";
16 echo "###. $trainer .###\n";
17 echo "###.          .###\n";
18 echo "#####\n";
19 ?>
```



- Kết quả thực thi đoạn code trên

```
root@22ce060f19b5:/var/www/html# php Trainer.php
#####
###.                .###
###. Trainer: Ash  .###
###.                .###
#####
root@22ce060f19b5:/var/www/html#
```

Một số Magic Methods khác

Magic Methods

Magic methods are special methods which override PHP's default's action when certain actions are performed on an object.

The following method names are considered magical: `__construct()`, `__destruct()`, `__call()`, `__callStatic()`, `__get()`, `__set()`, `__isset()`, `__unset()`, `__sleep()`, `__wakeup()`, `__serialize()`, `__unserialize()`, `__toString()`, `__invoke()`, `__set_state()`, `__clone()`, and `__debuginfo()`.

2.2. Kỹ thuật tấn công POP (Property Oriented Programming):

- Kỹ thuật này sẽ:
 - o Tận dụng Magic Methods.
 - o Thay đổi thuộc tính của các class.
- Can thiệp vào luồng hoạt động của chương trình.



- Ví dụ: Giả sử trong mã nguồn có 2 class như hình bên dưới:

```
class Database {
    private $connection;
    public function __destruct() {
        $this->connection->close();
    }
}
```

```
class Logger {
    private $filename;
    public function close() {
        unlink("/tmp/logs/" . $this->filename);
    }
}
```

- Tạo biến thuộc class `Logger` với `$filename` là đường dẫn chứa `"/tmp/logs/"` để tấn công Path Traversal.
- Tạo biến thuộc class `Database` rồi gán thuộc tính `connection` thành biến thuộc class `Logger` vừa được tạo
- Payload POP:

```
class Database {
    private $connection;
    public function __construct($conn) {
        $this->connection = $conn;
    }
}
class Logger {
    private $filename;
    public function __construct($filename) {
        $this->filename = $filename;
    }
}
$sourcePath = "../../../var/www/html/index.php";
$logger = new Logger($filename= $sourcePath);
$dbase = new Database($logger);
```



```
0:8:"Database":1:{
  s:20:"Databaseconnection";
  0:6:"Logger":1:{
    s:16:"Loggerfilename";
    s:28:"../../../var/www/html/index.php";
  }
}
```

Khi chương trình PHP kết thúc, hàm `__destruct()` của `Database` được gọi → `close()` của `Logger` được gọi với attack payload Path Traversal.



3. Tổng kết:

